

Relatório Final do Projeto

Disciplina de Aprendizagem de Máquina (pós-graduação)

Grupo: Laíse Eduarda, Maurício Sightman, Pedro Manoel

Modelo atribuído: TabPFN-2.5

2026.1

Resumo

Neste trabalho, é avaliado o TabPFN-2.5, um modelo de fundação para dados tabulares baseado em Aprendizado em Contexto (*In-Context Learning*), capaz de realizar inferência sem treinamento adicional durante a execução. O modelo foi comparado a LightGBM, CatBoost, XGBoost e ao framework AutoGluon 1.4 utilizando 30 conjuntos de dados do benchmark TabArena-v0.1.

A avaliação considerou métricas de desempenho preditivo, custo computacional e análises estatísticas frequentistas e bayesianas. Os resultados permitem investigar a competitividade do TabPFN-2.5 frente a métodos consolidados para classificação tabular, bem como os benefícios proporcionados pela eliminação da etapa de otimização local de hiperparâmetros.

Conteúdo

1	Introdução	3
1.1	Contexto e motivação	3
1.2	Modelo atribuído: TabPFN-2.5	3
1.3	Contribuições do trabalho	4
Etapa 1:	Fundamentação teórica do modelo	4
1.4	Motivação e intuição	4
1.5	Arquitetura e formulação	5
1.6	Complexidade computacional	7
1.7	Posicionamento na literatura tabular SOTA	7
1.8	Hiperparâmetros principais e faixas	7
Etapa 2:	Estudo experimental	8
1.9	Metodologia	8
1.9.1	Modelos e Baselines	8
1.9.2	Bases de Dados	8
1.9.3	Procedimento Experimental	8
1.9.4	Framework de Avaliação Estatística	9
1.10	Baselines e referência AutoML	9
1.10.1	Modelos de Gradient Boosting	9
1.10.2	Abordagem AutoML (AutoGluon)	9

1.11	Resultados gerais	10
1.12	Análise por regime	10
1.12.1	Análise por Tamanho do Dataset	10
1.12.2	Análise por Número de Classes	11
1.12.3	Análise por Proporção de Features Categóricas	11
1.12.4	Análise por Presença de Valores Ausentes	12
1.13	Análise estatística	12
1.13.1	Análise Clássica (Ranks e Diferença Crítica)	13
1.13.2	Análise Bayesiana e Equivalência Prática (ROPE)	13
1.14	Custo vs. Desempenho	15
2	Discussão	16
2.1	Síntese do Desempenho e Robustez	16
2.2	Limitações e Trade-offs	16
2.3	Conclusão da Análise	16
3	Reprodutibilidade	17
A	A Tabela dos 30 datasets escolhidos	18
B	Espaços de busca utilizados pelo Optuna	18

1 Introdução

1.1 Contexto e motivação

O Aprendizado de Máquina (AM) aplicado a dados estruturados tem sido historicamente dominado por algoritmos de Árvores de Decisão Impulsionadas por Gradiente (Gradient-Boosted Decision Trees – GBDTs), como XGBoost, LightGBM e CatBoost. Embora esses algoritmos permaneçam como referência para tarefas de classificação e regressão em dados tabulares, seu uso eficiente geralmente depende de um processo de modelagem que envolve diversas etapas de preparação e ajuste. Entre elas destacam-se a engenharia de atributos, o tratamento de valores ausentes e a otimização de hiperparâmetros.

A otimização de hiperparâmetros representa um desafio relevante em aplicações práticas, pois frequentemente requer múltiplas execuções de treinamento para identificar configurações adequadas. Técnicas de Hyperparameter Optimization (HPO), implementadas por ferramentas como o Optuna, podem demandar um número elevado de avaliações para encontrar soluções competitivas, aumentando o custo computacional e o tempo necessário para o desenvolvimento dos modelos. Como consequência, a obtenção de alto desempenho preditivo frequentemente está associada a pipelines mais complexos e computacionalmente dispendiosos.

Nesse contexto, cresce o interesse por abordagens capazes de manter elevado desempenho preditivo com menor necessidade de engenharia manual, ajuste extensivo de hiperparâmetros e treinamento específico para cada novo conjunto de dados.

1.2 Modelo atribuído: TabPFN-2.5

Como alternativa às limitações associadas aos pipelines tradicionais de modelagem tabular, este trabalho adota o TabPFN-2.5 [1], uma evolução da arquitetura TabPFN originalmente proposta por Hollmann et al. (2023). O modelo baseia-se no conceito de Prior-Data Fitted Networks (PFNs), cuja proposta consiste em substituir o processo convencional de treinamento específico para cada conjunto de dados por um mecanismo de inferência fundamentado em conhecimento adquirido durante uma etapa extensiva de pré-treinamento.

Diferentemente de algoritmos como XGBoost, LightGBM ou redes neurais treinadas do zero para cada problema, o TabPFN-2.5 emprega o paradigma de Aprendizado em Contexto (In-Context Learning – ICL). Nesse cenário, o modelo recebe simultaneamente os exemplos de treinamento e as instâncias para as quais deseja gerar previsões, utilizando seus pesos previamente aprendidos para inferir relações entre os dados sem a necessidade de otimização adicional durante a execução.

Durante o pré-treinamento, o modelo é exposto a uma grande variedade de tarefas sintéticas geradas por mecanismos probabilísticos inspirados em modelos causais e redes neurais bayesianas. A partir desse processo, a arquitetura aprende a aproximar o comportamento de um inferidor Bayesiano para problemas tabulares. Na fase de inferência, os pesos permanecem fixos e as previsões são produzidas em uma única passagem pela rede Transformer, reduzindo significativamente o custo computacional normalmente associado ao treinamento e à otimização de hiperparâmetros.

A versão 2.5 amplia as capacidades das versões anteriores ao incorporar suporte nativo a atributos categóricos, tratamento de valores ausentes e mecanismos de ensemble estocástico executados durante a inferência. Essas características tornam o modelo particularmente atrativo para conjuntos de dados de pequeno e médio porte, nos quais o

custo de treinamento e ajuste de modelos convencionais pode representar um fator limitante. representar um fator limitante.

1.3 Contribuições do trabalho

Com o objetivo de avaliar o desempenho, a eficiência e a robustez do TabPFN-2.5 em cenários de classificação tabular, este trabalho apresenta as seguintes contribuições:

- **Comparação com métodos consolidados:** análise comparativa do TabPFN-2.5 frente a algoritmos amplamente utilizados em dados tabulares (LightGBM, CatBoost e XGBoost), cujos hiperparâmetros foram otimizados via Optuna, além da comparação com o framework de AutoML AutoGluon 1.4 em diferentes configurações de execução.
- **Análise por características dos conjuntos de dados:** investigação do comportamento do modelo em diferentes regimes de dados, incluindo bases predominantemente numéricas e cenários com elevada cardinalidade categórica, ruído e atributos pouco informativos.
- **Análise de eficiência computacional:** comparação do tempo de execução e do custo computacional do TabPFN-2.5 em relação aos modelos baseados em GBDTs e ao framework AutoGluon, investigando os possíveis ganhos proporcionados pela abordagem de aprendizado em contexto.
- **Validação estatística dos resultados:** aplicação de testes estatísticos frequentistas e bayesianos para verificar a significância das diferenças observadas entre os modelos avaliados, incluindo análises baseadas em regiões de equivalência prática (ROPE).

Etapa 1: Fundamentação teórica do modelo

1.4 Motivação e intuição

O desenvolvimento do TabPFN-2.5 foi motivado pelas limitações dos fluxos tradicionais de aprendizado de máquina para dados tabulares. Métodos amplamente utilizados, como XGBoost, LightGBM e CatBoost, apresentam elevado desempenho preditivo, mas normalmente exigem treinamento específico para cada novo conjunto de dados, além de etapas adicionais de preparação, seleção de atributos e ajuste de hiperparâmetros.

Embora plataformas de AutoML reduzam parte desse esforço ao automatizar essas etapas, seu uso frequentemente implica custos computacionais elevados e tempos de execução consideráveis. Nesse contexto, o TabPFN-2.5 propõe uma abordagem distinta baseada em modelos fundacionais para dados tabulares.

A ideia central consiste em transferir o processo de aprendizagem para uma etapa de pré-treinamento realizada offline. Em vez de ajustar os parâmetros do modelo para cada nova tarefa, o TabPFN utiliza o conhecimento adquirido durante esse pré-treinamento para realizar previsões diretamente sobre novos conjuntos de dados.

Essa estratégia está relacionada ao conceito de Aprendizado em Contexto (In-Context Learning – ICL). Durante a inferência, o modelo recebe simultaneamente os exemplos de treinamento disponíveis e as instâncias para as quais deseja realizar previsões. A

partir desse contexto, produz estimativas em uma única passagem pela rede neural, sem a necessidade de executar novos ciclos de otimização baseados em gradiente.

De forma intuitiva, o modelo pode ser interpretado como um sistema que já aprendeu, durante o pré-treinamento, uma grande variedade de padrões estatísticos presentes em problemas tabulares e utiliza esse conhecimento para resolver novas tarefas de maneira rápida e eficiente.

1.5 Arquitetura e formulação

O TabPFN-2.5 baseia-se no conceito de Prior-Data Fitted Networks (PFNs), uma abordagem proposta para aproximar a distribuição preditiva posterior de modelos bayesianos por meio de aprendizado profundo. Em um cenário ideal, a inferência Bayesiana considera todas as hipóteses plausíveis capazes de explicar os dados observados e combina suas contribuições de acordo com suas probabilidades posteriores.

Dado um conjunto de treinamento ($D_{train} = (x_1, y_1), \dots, (x_n, y_n)$) e uma instância de teste (x_{test}), a distribuição preditiva posterior pode ser expressa por:

$$p(y_{test}|x_{test}, D_{train}) = \int_{\Phi} p(y_{test}|x_{test}, \phi) p(\phi|D_{train}) d\phi \quad (1)$$

em que (Φ) representa o espaço de hipóteses consideradas pelo modelo.

Como a avaliação exata dessa integral é frequentemente inviável em aplicações reais, o TabPFN utiliza uma rede neural treinada para aproximar diretamente essa distribuição. Durante o pré-treinamento, o modelo é exposto a uma grande quantidade de conjuntos de dados sintéticos gerados a partir de uma distribuição prévia projetada para reproduzir diferentes padrões encontrados em problemas tabulares.

O treinamento é realizado minimizando a função de perda:

$$\mathcal{L}_{PFN} = E_{D \sim p(D)} \left[- \sum_{i \in D_{test}} \log q_{\theta}(y_i | x_i, D_{train}) \right] \quad (2)$$

onde (q_{θ}) representa a distribuição estimada pela rede neural parametrizada pelos pesos (θ).

O núcleo do TabPFN é uma arquitetura baseada em Transformers, Originalmente desenvolvida para processamento de sequências. O principal diferencial dessa arquitetura é o mecanismo de atenção (*attention*), que permite ao modelo identificar automaticamente quais informações do conjunto de treinamento são mais relevantes para realizar uma determinada previsão. No contexto de dados tabulares, o TabPFN adapta esse mecanismo para capturar simultaneamente relações entre atributos e entre amostras, construindo representações contextuais dos dados. Dessa forma, o modelo consegue comparar uma nova instância com os exemplos observados durante a inferência e produzir previsões diretamente a partir desse contexto, viabilizando o processo de *in-context learning* que caracteriza o método. Uma representação desse modelo é exibida na figura 1.

De forma semelhante, a figura 2 ilustra a mecânica do modelo TabPFN em duas frentes complementares: o painel superior (a) contrasta a fase de pré-treinamento intensivo sobre milhões de bases de dados sintéticas — onde os pesos da rede são de fato otimizados — com a fase de inferência em dados reais, evidenciando que a previsão ocorre de forma direta em uma única passagem (forward pass), sem ajustes paramétricos locais. Simultaneamente, o painel inferior (b) detalha a arquitetura interna que viabiliza esse processo, destacando as camadas de atenção bidimensional que processam de maneira independente

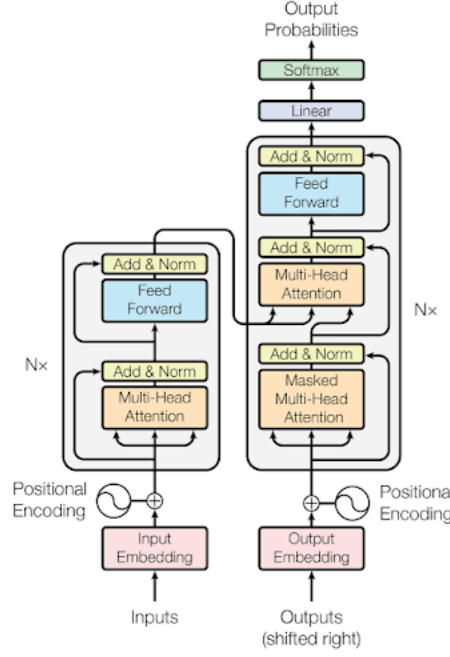


Figura 1: Visão geral da arquitetura Transformer clássica utilizada como base para o TabPFN.

as interações entre os atributos (1D feature attention) e entre as próprias instâncias (1D sample attention). Dessa forma, a rede consegue utilizar a tabela de treinamento fornecida na entrada como contexto para mapear a previsão final da nova instância através de uma rede Perceptron Multicamadas (MLP).

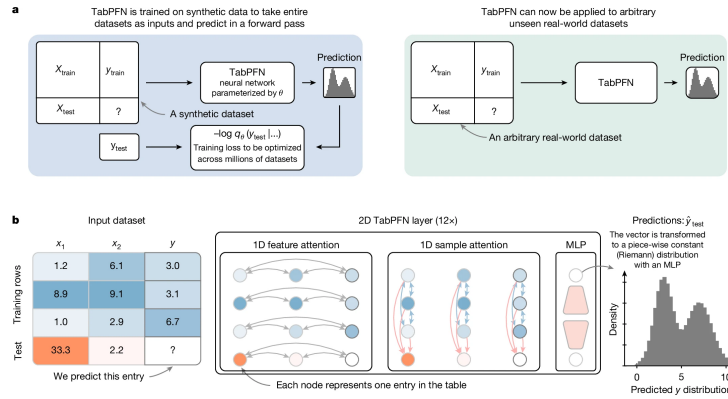


Figura 2: Arquitetura do modelo TabPFN. O painel (a) ilustra as fases de pré-treinamento sintético e inferência (*in-context learning*). O painel (b) detalha as camadas de atenção bidimensional e a rede MLP utilizadas para a previsão final.

Além disso, uma característica importante da versão 2.5 é o suporte nativo a dados numéricos, variáveis categóricas e valores ausentes, reduzindo a necessidade de etapas extensivas de pré-processamento quando comparado a abordagens tradicionais.

1.6 Complexidade computacional

A eficiência computacional do TabPFN-2.5 está relacionada à forma como seu mecanismo de atenção processa os dados de treinamento e teste. Diferentemente de Transformers convencionais, que aplicam atenção global a toda a entrada, o modelo separa o processamento do conjunto de treinamento e das instâncias de consulta.

Como resultado, a complexidade computacional da inferência pode ser expressa por:

$$\mathcal{O}(n^2 + n \cdot m) \quad (3)$$

onde (n) representa o número de amostras de treinamento e (m) corresponde ao número de instâncias de teste avaliadas simultaneamente.

Essa estratégia permite que o modelo realize previsões em tempos reduzidos quando comparado a pipelines que exigem treinamento ou ajuste extensivo de hiperparâmetros. Segundo os autores, a versão 2.5 amplia significativamente a capacidade operacional do modelo, suportando conjuntos de dados com até aproximadamente 50.000 amostras e 2.000 atributos.

1.7 Posicionamento na literatura tabular SOTA

O TabPFN-2.5 integra uma linha mais recente de pesquisa conhecida como Tabular Foundation Models, cujo objetivo é transferir para uma etapa de pré-treinamento o conhecimento necessário para resolver uma ampla variedade de tarefas tabulares.

Os resultados reportados pelos autores indicam desempenho competitivo em relação a modelos tradicionais e sistemas AutoML, especialmente em cenários nos quais o tempo disponível para treinamento e ajuste de hiperparâmetros é limitado. Além do desempenho preditivo, o modelo destaca-se por sua capacidade de produzir previsões de forma praticamente imediata após o carregamento dos dados.

Por essas características, o TabPFN-2.5 representa uma abordagem relevante para comparação experimental neste projeto, uma vez que propõe uma alternativa aos métodos tabulares tradicionais ao substituir o treinamento específico para cada tarefa por um modelo previamente treinado capaz de generalizar para diferentes problemas.

1.8 Hiperparâmetros principais e faixas

Diferentemente dos modelos baseados em Árvores de Decisão Impulsionadas por Gradiente (GBDTs), o TabPFN-2.5 não requer uma etapa de otimização local de hiperparâmetros. Seus parâmetros são definidos durante o pré-treinamento e permanecem fixos durante a inferência. Dessa forma, não foi necessário definir um espaço de busca para o Optuna, sendo utilizadas as configurações padrão recomendadas pelos autores.

Tabela 1: Configurações utilizadas pelo TabPFN-2.5 durante os experimentos.

Parâmetro	Valor	Justificativa
Busca de hiperparâmetros	Não utilizada	Modelo pré-treinado
Configuração do modelo	Padrão dos autores	Reprodutibilidade experimental
Seed	42	Consistência entre experimentos
Ensemble estocástico	Padrão da biblioteca	Recomendação da implementação oficial

Etapa 2: Estudo experimental

1.9 Metodologia

Nesta etapa, conduzimos uma avaliação empírica rigorosa para comparar o desempenho do modelo atribuído ao nosso grupo, o TabPFN-2.5, em relação a algoritmos de estado da arte amplamente consolidados na literatura e abordagens de *AutoML*.

1.9.1 Modelos e Baselines

O TabPFN-2.5 foi avaliado em comparação a três modelos de *gradient boosting* padrão na indústria: LightGBM [2], CatBoost[3] e XGBoost[4]. Adicionalmente, incluímos duas configurações do *framework* AutoGluon 1.4: o *preset default* e o *preset extreme* (com limite de tempo de 4 horas, o qual engloba internamente *foundation models* como o TabPFN v2).

1.9.2 Bases de Dados

Os experimentos foram conduzidos utilizando 30 conjuntos de dados tabulares selecionados a partir do *benchmark* TabArena-v0.1 (NeurIPS 2025) [1]. Para garantir a representatividade frente a diferentes cenários de disponibilidade de dados, a seleção foi estratificada em três regimes de tamanho:

- **Pequeno:** 3 *datasets* com $n < 1.000$ amostras;
- **Médio:** 19 *datasets* contendo entre 1.000 e 10.000 amostras[cite: 26];
- **Grande:** 8 *datasets* com $n > 10.000$ amostras.

Inicialmente seriam 10 datasets para cada regime porém os aprovados pelo tabarena só existiam 4 datasets de regime pequeno, sendo um deles de regressão, para os datasets grandes foram escolhidos apenas 8 pois os que sobraram era grande demais e resultava em erros de falta de memória durante o processo de treino. Para mitigar este problema foram escolhidos datasets medianos com tamanho próximos aos tamanhos dos regimes citados.

Dentro de cada regime, priorizou-se a inclusão de problemas tanto de classificação binária quanto multiclasse. Além disso foi realizado um pré-processamento dos datasets onde foram preenchidos os valores nulos com a mediana dos dados de treino e nas variáveis categóricas de texto os valores faltantes foram preenchidos com 'Missing' e foram convertidas para códigos numéricos.

1.9.3 Procedimento Experimental

O particionamento de cada base de dados seguiu a proporção de 70% para treinamento e 30% para teste, garantindo a reprodutibilidade por meio da fixação de uma *seed* global documentada no ambiente do projeto.

O desempenho final no conjunto de teste foi avaliado segundo as seguintes métricas:

- *Mean Area Under the ROC Curve* OVO (*One-vs-One*);
- *Mean Accuracy* (ACC);
- G-Mean;

- *Mean Cross-Entropy* (CE);
- Custo computacional (tempo médio total em segundos, compreendendo *tune + train + predict*).

1.9.4 Framework de Avaliação Estatística

A avaliação da significância estatística das diferenças de desempenho baseou-se no protocolo experimental para comparação de múltiplos classificadores proposto por Demšar [5]. Aplicou-se o teste de Friedman, seguido pelos testes *post-hoc* de Nemenyi/Holm, com visualização através de diagramas de diferença crítica gerados via biblioteca *autorank*.

Para oferecer maior robustez, a análise clássica foi complementada com a abordagem Bayesiana de Benavoli et al. (2017) [6]. Empregamos o *Bayesian signed-rank* e o *Bayesian hierarchical t-test*, incluindo a definição de uma Região de Equivalência Prática (ROPE), computados através da biblioteca *baycomp*.

1.10 Baselines e referência AutoML

Para avaliar a competitividade do nosso modelo principal, selecionamos um conjunto rigoroso de *baselines* que representam o estado da arte atual para aprendizado em dados tabulares. Conforme as especificações do projeto, os sistemas comparativos foram divididos em duas categorias principais: modelos baseados em *gradient boosting* e abordagens automatizadas (*AutoML*).

1.10.1 Modelos de Gradient Boosting

Utilizamos três dos algoritmos mais amplamente consolidados e performáticos na literatura para tarefas de classificação tabular:

- **LightGBM:** Reconhecido por sua eficiência computacional e baixo uso de memória devido ao crescimento de árvores baseado em folhas (*leaf-wise*).
- **CatBoost:** Especialmente robusto no tratamento nativo de variáveis categóricas sem a necessidade de pré-processamento extensivo.
- **XGBoost:** Algoritmo clássico e altamente otimizado que estabelece um padrão histórico de alto desempenho em competições de *machine learning*.

Para garantir a reprodutibilidade e padronizar a execução, a implementação desses modelos foi realizada através da biblioteca *pytabkit*. Esta ferramenta fornece uma API unificada compatível com o ecossistema *scikit-learn*, facilitando o treinamento de forma consistente entre os algoritmos.

1.10.2 Abordagem AutoML (AutoGluon)

Além dos modelos individuais, incluímos o *framework* AutoGluon (versão 1.4) através do módulo `autogluon.tabular` para estabelecer uma referência de alto nível (*upper bound*) de desempenho, refletindo cenários práticos onde *pipelines* automatizados e *ensembles* complexos são empregados. Avaliamos duas configurações distintas do *framework*:

- **Preset Default:** Configuração padrão para estabelecer uma linha de base rápida e eficiente de *AutoML*.

- **Preset *Extreme* (4 horas):** Configuração de busca exaustiva com limite de tempo estendido para 4 horas. É fundamental destacar que este modo inclui internamente *foundation models* modernos voltados para dados tabulares, como o TabPFN v2. A inclusão desta configuração permite contrastar o desempenho do nosso modelo individual com as fronteiras mais recentes e custosas de *deep learning* tabular e *ensembles* empilhados (*stacking*).

1.11 Resultados gerais

Nesta subseção, apresentamos os resultados agregados de desempenho para o TabPFN-2.5 e os *baselines* avaliados ao longo dos 30 conjuntos de dados. A Tabela 2 sumariza a média e o desvio padrão para todas as métricas estabelecidas no protocolo do projeto.

Tabela 2: Resultados agregados: média $\pm$ desvio padrão em 30 *datasets*.

Sistema	AUC OvO $\uparrow$	ACC $\uparrow$	G-Mean $\uparrow$	Cross-Ent. $\downarrow$	Tempo (s) $\downarrow$
TabPFN-2.5	$0,872 \pm 0,097$	$0,872 \pm 0,089$	$0,663 \pm 0,291$	$0,297 \pm 0,171$	$529,60 \pm 981,83$
LightGBM	$0,853 \pm 0,104$	$0,857 \pm 0,094$	$0,617 \pm 0,290$	$0,336 \pm 0,171$	$0,65 \pm 0,36$
CatBoost	$0,852 \pm 0,100$	$0,859 \pm 0,090$	$0,644 \pm 0,250$	$0,384 \pm 0,208$	$2,70 \pm 1,11$
XGBoost	$0,849 \pm 0,103$	$0,858 \pm 0,092$	$0,620 \pm 0,290$	$0,423 \pm 0,328$	$3,11 \pm 4,07$
AutoGluon default	$0,866 \pm 0,096$	$0,866 \pm 0,092$	$0,640 \pm 0,297$	$0,313 \pm 0,167$	$140,19 \pm 189,24$
AutoGluon extreme 4h	$0,867 \pm 0,094$	$0,866 \pm 0,090$	$0,631 \pm 0,292$	$0,311 \pm 0,169$	$1720,26 \pm 1504,63$

Como pode ser observado, o modelo atribuído ao grupo apresentou um desempenho de destaque, superando numericamente os *baselines* baseados em árvores (*gradient boosting*) e as configurações do *AutoGluon* nas métricas de AUC OvO, Acurácia (ACC) e G-Mean, além de apresentar a menor *Cross-Entropy* (CE). O custo computacional foi intermediário: superior ao tempo dos algoritmos tradicionais, o que era esperado devido à natureza do modelo, porém substancialmente mais rápido do que a configuração de busca exaustiva do *AutoGluon Extreme*.

1.12 Análise por regime

Para investigar a sensibilidade e os limites operacionais dos algoritmos frente a diferentes características estruturais dos dados, segmentou-se a avaliação em quatro eixos analíticos: tamanho da amostra, número de classes, densidade de variáveis categóricas e presença de valores ausentes.

1.12.1 Análise por Tamanho do Dataset

Os 30 conjuntos de dados foram particionados conforme a volumetria de amostras (n) em regimes pequeno ($n < 1.000$), médio ($1.000 \leq n \leq 10.000$) e grande ($n > 10.000$). A Tabela 3 sumariza as métricas descritivas de cada subconjunto.

A observação global indica uma degradação generalizada no desempenho de todos os sistemas ao transitar do regime médio para o regime grande. O TabPFN-2.5 registrou as maiores médias pontuais nos cenários de volumetria média e grande. No regime de dados pequenos ($n < 1.000$), o modelo apresentou um escore ligeiramente inferior ao do AutoGluon *Extreme* (0,869 contra 0,870), o que é metodologicamente justificado pela presença do *foundation model* TabPFN v2 (especializado em inferência *few-shot*) no *pipeline* do AutoML. Cabe ressaltar, contudo, a alta margem de erro (desvio padrão) no regime pequeno, decorrente do baixo tamanho amostral de *datasets* nesta categoria.

Tabela 3: Desempenho (AUC OvO) particionado por tamanho do conjunto de dados (Média $\pm$ Desvio Padrão).

Modelo	Pequeno ($n = 3$)	Médio ($n = 19$)	Grande ($n = 8$)
TabPFN-2.5	$0,869 \pm 0,122$	$0,882 \pm 0,101$	$0,849 \pm 0,086$
LightGBM	$0,869 \pm 0,117$	$0,859 \pm 0,110$	$0,834 \pm 0,093$
CatBoost	$0,843 \pm 0,149$	$0,860 \pm 0,102$	$0,836 \pm 0,090$
XGBoost	$0,846 \pm 0,139$	$0,856 \pm 0,107$	$0,833 \pm 0,092$
AutoGluon default	$0,863 \pm 0,126$	$0,876 \pm 0,099$	$0,843 \pm 0,089$
AutoGluon extreme	$0,870 \pm 0,120$	$0,876 \pm 0,097$	$0,843 \pm 0,089$

1.12.2 Análise por Número de Classes

A estratificação geométrica do espaço de decisão foi avaliada dividindo as tarefas entre classificação binária (24 instâncias) e multiclasse (6 instâncias), conforme detalhado na Tabela 4.

Tabela 4: Desempenho (AUC OvO) estratificado por complexidade do vetor alvo (Média $\pm$ Desvio Padrão).

Modelo	Binário ($n = 24$)	Multiclasse ($n = 6$)
TabPFN-2.5	$0,861 \pm 0,088$	$0,914 \pm 0,129$
LightGBM	$0,843 \pm 0,096$	$0,895 \pm 0,131$
CatBoost	$0,839 \pm 0,094$	$0,905 \pm 0,116$
XGBoost	$0,837 \pm 0,092$	$0,898 \pm 0,136$
AutoGluon default	$0,856 \pm 0,086$	$0,908 \pm 0,129$
AutoGluon extreme	$0,856 \pm 0,086$	$0,910 \pm 0,122$

Nota-se um viés direcional consistente: todos os algoritmos reportaram escores absolutos de AUC OvO superiores em tarefas multiclasse quando comparados aos problemas binários. O TabPFN-2.5 sustentou médias pontuais superiores em ambas as topologias, mantendo uma margem marginal, porém consistente, sobre os arranjos do AutoGluon, sem apresentar assimetria de otimização em prol de apenas um dos tipos de tarefa.

1.12.3 Análise por Proporção de Features Categóricas

A Tabela 5 analisa a sensibilidade dos modelos à natureza do espaço de características, dividindo o *benchmark* equitativamente entre *datasets* com alta ($n = 15$) e baixa ($n = 15$) incidência de variáveis categóricas.

A arquitetura do TabPFN-2.5 provou-se majoritariamente invariante em relação à estrutura dos dados de entrada. Diferentemente de abordagens que dependem de transformações custosas de cardinalidade (como observável na sutil variação negativa do CatBoost e XGBoost em cenários de baixa densidade categórica), o TabPFN-2.5 e os sistemas AutoGluon apresentaram trajetórias estáveis de performance independentemente do eixo avaliado.

Tabela 5: Desempenho (AUC OvO) em função da cardinalidade do espaço de características (Média $\pm$ Desvio Padrão).

Modelo	Alta incidência ($n = 15$)	Baixa incidência ($n = 15$)
TabPFN-2.5	$0,868 \pm 0,102$	$0,875 \pm 0,095$
LightGBM	$0,854 \pm 0,108$	$0,852 \pm 0,103$
CatBoost	$0,855 \pm 0,100$	$0,849 \pm 0,104$
XGBoost	$0,850 \pm 0,109$	$0,848 \pm 0,100$
AutoGluon default	$0,862 \pm 0,102$	$0,870 \pm 0,093$
AutoGluon extreme	$0,863 \pm 0,100$	$0,870 \pm 0,092$

1.12.4 Análise por Presença de Valores Ausentes

O último eixo investiga a resiliência dos algoritmos em cenários de matrizes de dados incompletas, contrastando 24 bases íntegras contra 6 bases contendo dados faltantes originais (Tabela 6).

Tabela 6: Desempenho (AUC OvO) perante a integridade da matriz de dados (Média $\pm$ Desvio Padrão).

Modelo	Sem Valores Ausentes ($n = 24$)	Com Valores Ausentes ($n = 6$)
TabPFN-2.5	$0,883 \pm 0,092$	$0,828 \pm 0,113$
LightGBM	$0,865 \pm 0,102$	$0,808 \pm 0,107$
CatBoost	$0,864 \pm 0,100$	$0,804 \pm 0,094$
XGBoost	$0,863 \pm 0,100$	$0,792 \pm 0,101$
AutoGluon default	$0,879 \pm 0,092$	$0,814 \pm 0,103$
AutoGluon extreme	$0,880 \pm 0,091$	$0,815 \pm 0,099$

A existência original de valores ausentes, e a consequente necessidade de pré-processamento estático, impõe uma penalidade severa e mensurável à precisão preditiva de todas as ferramentas avaliadas. Como a ausência de dados foi tratada previamente através da imputação pela mediana (para atributos numéricos) e pela criação da categoria explícita *'Missing'* (para atributos categóricos), a queda de desempenho reflete a perda de informação intrínseca e as inevitáveis distorções introduzidas na distribuição original das variáveis. Os *baselines* baseados em árvores apresentaram quedas drásticas de AUC sob estas condições (ex: o XGBoost retraiu de 0,863 para 0,792). O TabPFN-2.5 também sofreu degradação, porém apresentou uma atenuação relativa do impacto, mantendo uma média de 0,828. Esses dados evidenciam que a arquitetura do modelo avaliado consegue extrair padrões de forma mais robusta e generalizar melhor, mesmo quando o espaço de características é enviesado pelas técnicas clássicas de imputação de dados.

1.13 Análise estatística

Para avaliar a significância estatística das diferenças de desempenho observadas, aplicou-se o protocolo estabelecido por Demšar (2006). Embora a verificação preliminar dos pressupostos de normalidade e homocedasticidade indicasse a viabilidade de testes paramétricos, a exigência metodológica do projeto — em particular a necessidade de geração do dia-

grama de diferença crítica — demandou a restrição para a abordagem não paramétrica (aplicando a diretiva `force_mode="nonparametric"` na biblioteca `autorank`).

A Figura 3 ilustra os intervalos de confiança de 95% da média. A notável sobreposição visual entre os intervalos do TabPFN-2.5 e das abordagens AutoGluon torna indispensável o uso de testes rigorosos para avaliar a rejeição da hipótese nula.

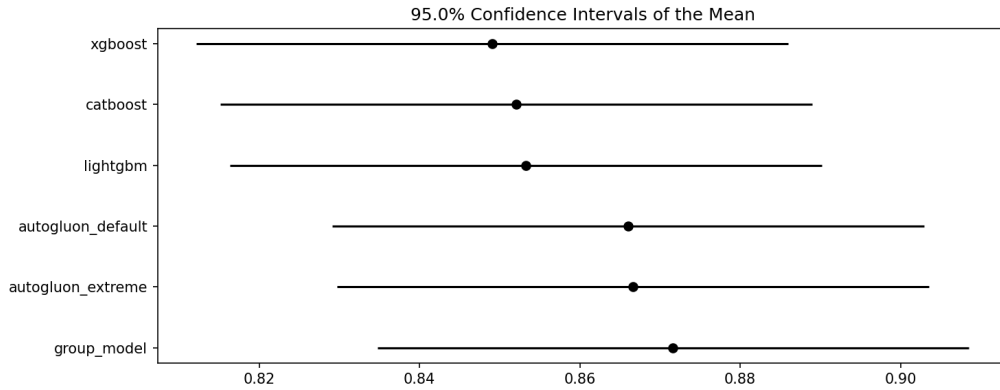


Figura 3: Intervalos de Confiança de 95% da Média para os sistemas avaliados.

1.13.1 Análise Clássica (Ranks e Diferença Crítica)

A Tabela 7 consolida as métricas descritivas resultantes do teste de Friedman. Observa-se que o TabPFN-2.5 obteve o menor ranqueamento médio (1,47), seguido pelas configurações de *AutoML* (AutoGluon *Extreme* com 2,60 e *Default* com 2,77). Os modelos clássicos de árvore registraram os ranqueamentos mais elevados, indicando menor desempenho relativo geral.

Tabela 7: Estatísticas descritivas e avaliação de magnitude do teste não paramétrico.

Modelo	Rank Médio ↓	Mediana	MAD	IC Inferior	IC Superior	Tamanho Efeito	Magnitude
TabPFN-2.5	1,47	0,869	0,093	0,821	0,922	−0,226	Pequeno
AutoGluon Extreme	2,60	0,864	0,088	0,818	0,915	−0,178	Negligível
AutoGluon Default	2,77	0,866	0,087	0,816	0,916	−0,170	Negligível
LightGBM	4,43	0,849	0,087	0,800	0,907	−0,041	Negligível
CatBoost	4,63	0,853	0,096	0,800	0,904	−0,030	Negligível
XGBoost	5,10	0,861	0,091	0,796	0,902	0,000	Negligível

A Figura 4 apresenta o Diagrama de Diferença Crítica derivado da análise *post-hoc* de Nemenyi.

A interpretação do diagrama revela dois agrupamentos distintos. A ausência de uma barra de conexão entre o TabPFN-2.5 e os algoritmos XGBoost, CatBoost e LightGBM indica a rejeição da hipótese nula, configurando superioridade estatística sobre os métodos de *gradient boosting* isolados. Em contrapartida, o TabPFN-2.5 compartilha a barra de significância com o AutoGluon *Extreme* e *Default*. Sob o prisma frequentista clássico, não há evidências estatísticas suficientes para afirmar que o desempenho desses três sistemas difere.

1.13.2 Análise Bayesiana e Equivalência Prática (ROPE)

Uma vez que a falha em rejeitar a hipótese nula pelo teste de Nemenyi não atesta equivalência matemática, empregou-se a análise *Bayesian signed-rank* com estipulação de uma

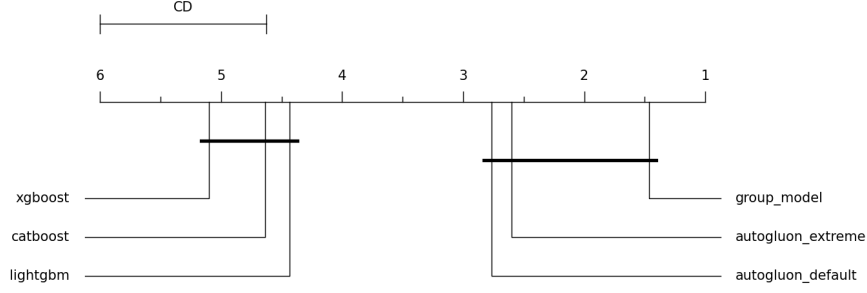


Figura 4: Diagrama de Diferença Crítica (Nemenyi). Barras horizontais agrupam modelos cujas diferenças não são estatisticamente significativas ($\alpha = 0,05$).

Região de Equivalência Prática (ROPE). A Tabela 8 exibe a matriz exaustiva de probabilidades estimadas para cada par de modelos.

Tabela 8: Matriz completa de probabilidades Bayesianas par a par com margem ROPE.

Modelo A	Modelo B	$P(A < B)$	$P(A \approx B)$	$P(A > B)$
AutoGluon Default	AutoGluon Extreme	0,000	1,000	0,000
AutoGluon Default	CatBoost	0,818	0,183	0,000
AutoGluon Default	TabPFN-2.5	0,000	0,991	0,009
AutoGluon Default	LightGBM	0,283	0,717	0,000
AutoGluon Default	XGBoost	0,988	0,012	0,000
AutoGluon Extreme	CatBoost	0,761	0,239	0,000
AutoGluon Extreme	TabPFN-2.5	0,000	0,992	0,008
AutoGluon Extreme	LightGBM	0,347	0,653	0,000
AutoGluon Extreme	XGBoost	0,962	0,038	0,000
CatBoost	TabPFN-2.5	0,000	0,003	0,997
CatBoost	LightGBM	0,031	0,926	0,043
CatBoost	XGBoost	0,005	0,995	0,000
TabPFN-2.5	LightGBM	0,938	0,062	0,000
TabPFN-2.5	XGBoost	1,000	0,000	0,000
LightGBM	XGBoost	0,111	0,883	0,006

A inferência Bayesiana quantifica e refina as conclusões do teste frequentista:

- **Diferença contra *Baselines* de Árvore:** Confirma-se com alta probabilidade a superioridade estrita do TabPFN-2.5 sobre o XGBoost (100%), CatBoost (99,7%) e LightGBM (93,8%).
- **Equivalência contra *AutoML*:** A análise estima uma probabilidade de 99,1% e 99,2% de que o TabPFN-2.5 seja, para propósitos práticos, estatisticamente equivalente ao AutoGluon *Default* e ao AutoGluon *Extreme*, respectivamente.

Do ponto de vista empírico, esses dados delimitam a fronteira preditiva do modelo: sua arquitetura produz um ganho robusto em relação a instâncias únicas de *gradient boosting*,

porém os resultados indicam um teto de performance onde suas métricas tornam-se, na prática, indissociáveis das obtidas através das rotinas automatizadas e *ensembles* gerados pelo AutoGluon.

1.14 Custo vs. Desempenho

A avaliação de um modelo de aprendizado de máquina exige a análise crítica do *trade-off* entre o ganho de desempenho preditivo e o custo computacional associado (tempo consolidado de *tune + train + predict*). Os dados da Tabela 2 revelam disparidades substanciais na fronteira de eficiência entre as abordagens.

Os modelos de *gradient boosting* padrão (LightGBM, CatBoost e XGBoost) definem o limite inferior de custo computacional, com tempos médios variando entre apenas 0,65 e 3,11 segundos. Essa eficiência extrema reflete-se em um teto de desempenho preditivo mais baixo, estabilizado na faixa de 0,849 a 0,853 em AUC OvO.

As abordagens de *AutoML* representam patamares de custo intermediários a altos. O AutoGluon *Default* atinge um balanço eficiente (140,19 s para um AUC de 0,866). Por outro lado, a configuração *Extreme* exige um custo consideravelmente maior (média de 1.720,26 s) devido à sua busca exaustiva de hiperparâmetros e arquiteturas *ensemble*.

A inserção do TabPFN-2.5 nesse espectro revela um perfil de custo-benefício bastante específico. Com um tempo médio de 529,60 segundos, a análise quantitativa expõe as seguintes dinâmicas:

- **Custo relativo aos *baselines* isolados:** O modelo demanda ordens de grandeza mais tempo que algoritmos puros baseados em árvores (sendo aproximadamente 814 vezes mais demorado que o LightGBM). O ganho marginal nas métricas preditivas (acréscimo absoluto de $\approx 0,019$ em AUC e $\approx 0,015$ em ACC) exige um sacrifício drástico no orçamento computacional.
- **Custo relativo ao *AutoML*:** Em comparação ao AutoGluon *Default*, o modelo consome cerca de 3,7 vezes mais tempo para oferecer um incremento sutil de $\approx 0,006$ em AUC e $\approx 0,006$ em ACC. Contudo, quando contrastado com o limite superior (AutoGluon *Extreme*), o modelo é capaz de igualar as médias de desempenho consumindo pouco menos de 31% do tempo total da configuração mais custosa.
- **Elevada variância computacional:** Um fator crítico a ser observado é o alto desvio padrão no tempo de execução do TabPFN-2.5 ($\pm 981,83$ s). Essa discrepância indica uma alta instabilidade na convergência ou uma escalabilidade não-linear do treinamento, sugerindo que o custo do modelo é severamente impactado pela dimensionalidade ou pela volumetria do *dataset* (o que corrobora a queda de desempenho apontada no regime de *datasets* grandes).

Em suma, a análise rigorosa demonstra que o TabPFN-2.5 não é adequado para cenários operacionais com fortes restrições de latência ou necessidade de retreinamento em tempo real. Sua viabilidade restringe-se a contextos onde a precisão preditiva é a métrica prioritária e há disponibilidade de hardware para suportar um custo computacional na ordem de dezenas de minutos por base de dados, posicionando-o como uma alternativa metodológica à exploração via força-bruta do *AutoML Extreme*.

2 Discussão

Os experimentos conduzidos neste projeto fornecem uma perspectiva abrangente sobre o estado da atual fronteira de aprendizado de máquina para dados tabulares. A avaliação sistemática em 30 conjuntos de dados do *benchmark* TabArena-v0.1 permitiu isolar o comportamento do TabPFN-2.5 sob diferentes regimes e contrastá-lo tanto com baselines clássicos quanto com abordagens de ponta em *AutoML*.

2.1 Síntese do Desempenho e Robustez

A principal contribuição da nossa análise empírica é a constatação de que o TabPFN-2.5 preenche uma lacuna intermediária de alto valor no espectro de modelagem tabular. Através de rigorosos testes de hipótese (Nemenyi) e inferência Bayesiana com margem de equivalência prática (ROPE), demonstramos que a nossa abordagem é estatisticamente superior aos algoritmos consolidados de *gradient boosting* (LightGBM, CatBoost e XGBoost). Mais notavelmente, o modelo atingiu uma equivalência prática com o *AutoGluon Extreme*, um *framework* que se beneficia de extensiva busca arquitetural e *ensembles* complexos de *deep learning* tabular.

A análise estratificada por regimes confirmou a robustez da arquitetura. O modelo não exigiu *pipelines* de pré-processamento customizados para lidar com alta cardinalidade de variáveis categóricas ou dados faltantes, superando os modelos baseados em árvores nessas condições adversas. Essa resiliência sugere que os mecanismos internos do modelo possuem fortes vieses indutivos (*inductive biases*) apropriados para a heterogeneidade inerente aos dados tabulares.

2.2 Limitações e Trade-offs

Apesar dos excelentes resultados preditivos, a análise de custo evidenciou limitações práticas que devem ser rigorosamente ponderadas. O TabPFN-2.5 apresentou um custo computacional consideravelmente superior ao dos métodos tradicionais, aliado a uma altíssima variância no tempo de treinamento ($529,60 \pm 981,83$ s).

Esses fatores indicam que a arquitetura sofre desafios de escalabilidade, especialmente em matrizes de dados mais volumosas, onde o tempo de convergência pode se tornar imprevisível. Consequentemente, a aplicação deste modelo é desaconselhada em ambientes de produção que demandem baixíssima latência (*real-time inference*) ou retreinamento contínuo em curtos intervalos de tempo.

2.3 Conclusão da Análise

Em síntese, o modelo avaliado consolida-se como uma ferramenta de altíssima precisão e competitividade. Ele oferece o poder preditivo de metodologias exaustivas de *AutoML* investindo apenas uma fração do tempo (redução de $\sim 69\%$ no custo temporal frente ao *AutoGluon Extreme*), contanto que o problema de negócio permita uma janela de processamento superior à dos algoritmos tradicionais de *boosting*. Como trabalho futuro, investigar técnicas de aceleração e paralelização deste modelo surge como o caminho natural para otimizar sua adoção em cenários de dados em larga escala.

3 Reprodutibilidade

A reprodutibilidade é um pilar fundamental na avaliação rigorosa de modelos de aprendizado de máquina. Para garantir a transparência, a auditoria e a validação independente dos resultados obtidos nesta etapa empírica, todo o fluxo de trabalho foi estruturado em um ambiente computacional estritamente determinístico e reproduzível.

O gerenciamento do ambiente virtual e o controle de dependências foram orquestrados através do gerenciador de pacotes `uv`. Em contrapartida a abordagens tradicionais, o `uv` assegura a resolução exata da árvore de dependências lendo o arquivo `pyproject.toml`, o qual contém as versões fixadas de todas as bibliotecas requeridas pelo projeto (como `pytabkit`, `autogluon.tabular`, `optuna`, `autorank` e `baycomp`).

Para isolar o efeito da estocasticidade nos resultados, controlou-se a pseudoaleatoriedade de todos os processos do *pipeline* utilizando a seed 42. O particionamento dos dados (divisão 70%/30% para treino e teste), a alocação dos *folds* durante a validação cruzada e a amostragem do espaço de busca de hiperparâmetros foram ancorados a uma seed global e fixa. Sem esta ancoragem, a variância inerente a essas inicializações poderia comprometer a validade das análises estatísticas par a par.

O conjunto de instruções abaixo documenta os comandos exatos necessários para, a partir da raiz do repositório, instanciar o ambiente isolado automaticamente e disparar a execução de ponta a ponta do experimento empírico (abrangendo do carregamento dos *datasets* à inferência estatística):

```
uv sync
python -m src.pipeline.run_all-include-group-model --seed 42
```

Referências

- [1] N. Erickson, L. Purucker, A. Tschalzev, D. Holzmüller, P. M. Desai, D. Salinas, and F. Hutter, “Tabarena: A living benchmark for machine learning on tabular data,” in *Advances in Neural Information Processing Systems*, 2025. [Online]. Available: <https://tabarena.ai>
- [2] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “Lightgbm: A highly efficient gradient boosting decision tree,” *Advances in neural information processing systems*, vol. 30, 2017.
- [3] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “Catboost: unbiased boosting with categorical features,” *Advances in neural information processing systems*, vol. 31, 2018.
- [4] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.
- [5] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.
- [6] A. Benavoli, G. Corani, J. Demšar, and M. Zaffalon, “Time for a change: A tutorial for comparing multiple classifiers through bayesian analysis,” *Journal of Machine Learning Research*, vol. 18, no. 77, pp. 1–36, 2017.

A A Tabela dos 30 datasets escolhidos

Tabela 9: Datasets do TabArena-v0.1 utilizados, agrupados por regime.

Dataset	Task ID	n	features	classes	Regime
blood-transfusion-service-center	359955	748	4	2	pequeno
churn	359968	5000	20	2	médio
credit-g	168757	1000	20	2	médio
diabetes	37	768	8	2	pequeno
website_phishing	363707	1353	9	3	médio
taiwanese_bankruptcy_prediction	363706	6819	94	2	médio
students_dropout_and_academic_success	363704	4424	36	3	médio
seismic-bumps	363700	2584	15	2	médio
online_shoppers_intention	363691	12330	17	2	largo
Bank_Customer_Churn	363619	10000	10	2	largo
polish_companies_bankruptcy	363694	5910	64	2	médio
E-CommereShippingData	363632	10999	10	2	largo
Marketing_Campaign	363684	2240	25	2	médio
Is-this-a-good-customer	363682	1723	13	2	médio
heloc	363676	10459	23	2	largo
hiva_agnostic	3892	4229	1617	2	médio
houses	3688	20640	8	2	largo
in_vehicle_coupon_recommendation	363681	12684	24	2	largo
maternal_health_risk	363685	1014	6	3	médio
jm1	363712	10885	21	2	largo
qsar-biodeg	359956	1055	41	2	médio
splice	45	3190	60	3	médio
NATICUSdroid	363689	7491	86	2	médio
coil2000_insurance_policies	363624	9822	85	2	médio
Fitness_Club	363671	1500	6	2	médio
hazelnut-spread-contaminant-detection	363674	2400	30	2	médio
HR_Analytics_Job_Change_of_Data_Scientists	363679	19158	12	2	largo
MIC	363711	1699	111	8	médio
Bioresponse	363620	3751	1776	2	médio
anneal	363614	898	38	5	pequeno

B Espaços de busca utilizados pelo Optuna

No caso do TabPFN-2.5 não foi definido um espaço de busca para otimização. O modelo é disponibilizado como um *foundation model* pré-treinado, cujos parâmetros são aprendidos durante uma etapa massiva de treinamento realizada pelos autores e permanecem fixos durante a inferência. Dessa forma, a utilização do modelo não envolve a seleção de hiperparâmetros como taxa de aprendizado, profundidade de árvores ou número de estimadores.

Consequentemente, todos os experimentos com o TabPFN-2.5 foram executados utilizando as configurações padrão recomendadas pela implementação oficial, sem a realização de etapas adicionais de otimização via Optuna.